



Intro to Reinforcement Learning

强化学习纲要

Lecture 10: RL in a nutshell

Bolei Zhou

<https://github.com/zhoubolei/introRL>

Intro to RL: Finale



Lecture 1	Overview
Lecture 2	Markov Decision Process
Lecture 3	Model-free Prediction and Control
Lecture 4	Value Function Approximation
Lecture 5	Policy Optimization: Foundation
Lecture 6	Policy Optimization: State of the art
Lecture 7	Model-based RL
Lecture 8	Imitation Learning
Lecture 9	Distributed systems for RL
Lecture 10	RL in a nutshell

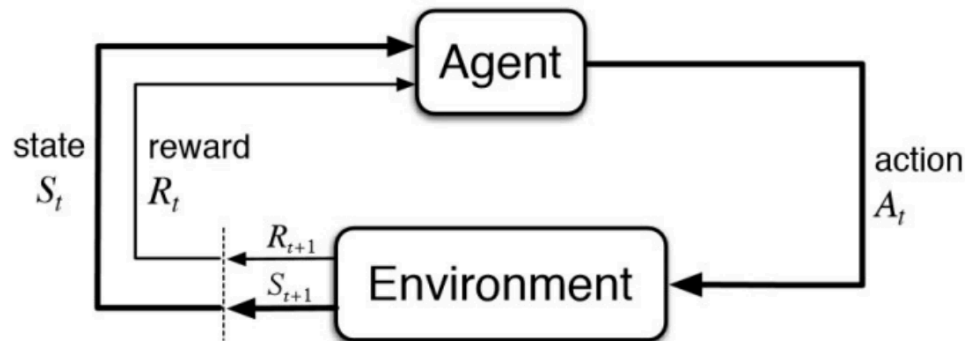
Outline

- Basics of RL
- MDP and tabular solution methods
- Value function approximation
- Policy optimization
- Model-based RL, imitation learning, RL system design
- Open problems

Basics of RL

Definition of RL:

a computational approach to learning whereby **an agent** tries to maximize the total amount of **reward** it receives while **interacting** with a complex and uncertain **environment**.



Basics of RL

Properties of RL

- Sequential data as input (not i.i.d)
- Trial and error exploration (balance between exploration and exploitation)
- Actions may have long-term consequence (rewards are delayed)

Basics of RL

An RL agent includes one or more of these components:

- **Policy function**: how action is selected
 - Stochastic policy: probability over actions (20% take LEFT, 80% take RIGHT)
 - Deterministic policy: take action LEFT

- **Value function**: how good is each state of action

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s] = E_{\pi}[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} | S_t = s]$$

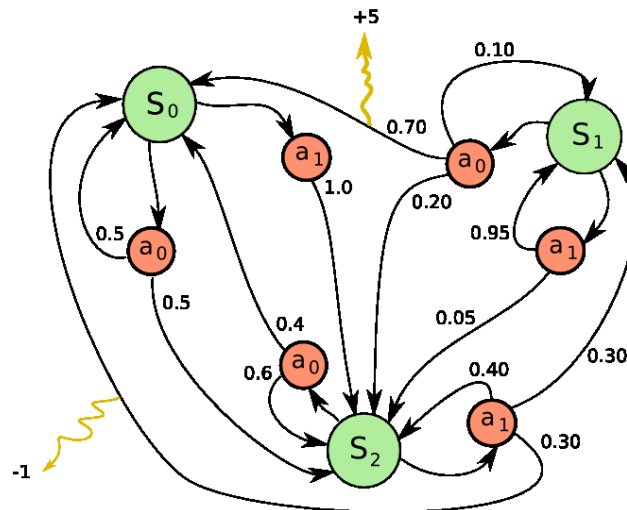
- **Model**: state transition of the environment

$$P_{ss'}^a = \mathbb{P}[S_{t+1} = s' | S_t = s, A_t = a]$$

Markov Decision Processes (MDPs)

Definition of MDP

- 1 P^a is dynamics/transition model for each action
 $P(S_{t+1} = s' | S_t = s, A_t = a)$
- 2 $\mathcal{R}$ is a reward function $R(S_t = s, A_t = a) = \mathbb{E}[R_t | S_t = s, A_t = a]$
- 3 Discount factor $\gamma \in [0, 1]$



Bellman Expectation Equation

- State-value function can be decomposed into immediate reward plus discounted value of the successor state,

$$V_{\pi}(s) = E_{\pi}[R_{t+1} + \gamma V_{\pi}(s_{t+1}) | s_t = s]$$

- Action-value function can similarly be decomposed

$$Q_{\pi}(s, a) = E_{\pi}[R_{t+1} + \gamma Q_{\pi}(s_{t+1}, A_{t+1}) | s_t = s, A_t = a]$$

Evaluation and Control in MDP

- Evaluation: Given an MDP, evaluate the value of a policy
- Control: Given an MDP, find the optimal policy which leads to optimal value

MDP Policy Evaluation

- ① Problem: Evaluate a given policy π for a MDP
- ② Output the value function under policy v_π
- ③ Solution: iteration on Bellman expectation backup
- ④ Synchronous backup algorithm:
 - ① At each iteration $k+1$
Bellman expectation backup on MDP

$$v_{k+1}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) (R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) v_k(s'))$$

Or in the form of Bellman expectation backup on MRP
 $\langle \mathcal{S}, \mathcal{P}^\pi, \mathcal{R}^\pi, \gamma \rangle$

$$v_{k+1}(s) = R^\pi(s) + \gamma P^\pi(s'|s) v_k(s')$$

MDP Control

- Objective: Find the optimal policy, given a MDP
- Policy iteration and value iteration

- 1 Problem: Find the optimal policy π
- 2 Solution: iteration on the Bellman optimality backup
- 3 Value Iteration algorithm:
 - 1 initialize $k = 1$ and $v_0(s) = 0$ for all states s
 - 2 For $k = 1 : H$
 - 1 for each state s

$$q_{k+1}(s, a) = R(s, a) + \gamma \sum_{s' \in S} P(s'|s, a) v_k(s')$$

$$v_{k+1}(s) = \max_a q_{k+1}(s, a)$$

- 2 $k \leftarrow k + 1$

- 4 To retrieve the optimal policy after the value iteration:

$$\pi(s) = \arg \max_a R(s, a) + \gamma \sum_{s' \in S} P(s'|s, a) v_{k+1}(s')$$

<https://github.com/zhoubolei/RExample/tree/master/MDP>

Model-free prediction and control

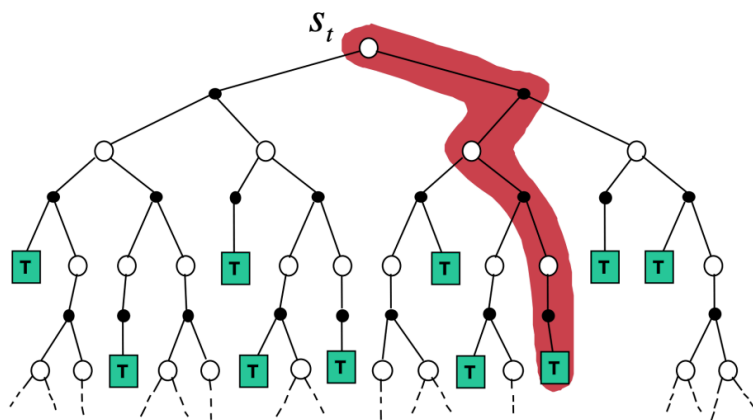
- Previously we assume R and P of the MDP are given so we can compute the state-action value of a policy

$$q_{\pi_i}(s, a) = R(s, a) + \gamma \sum_{s' \in S} P(s'|s, a) v_{\pi_i}(s')$$

- What to do if neither of them are unknown (model-free)
 - Monte Carlo (MC): Simulation-based
 - Temporal Difference (TD): Bootstrapping

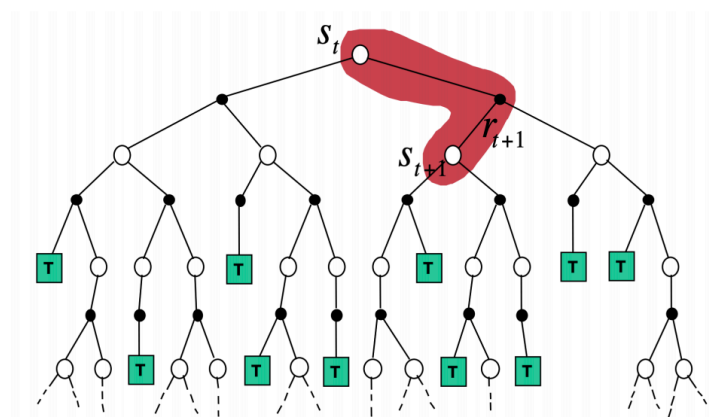
Policy Evaluation using Monte-Carlo and Temporal Difference

$$v(S_t) \leftarrow v(S_t) + \alpha(G_t - v(S_t))$$



Monte-Carlo Simulation

$$TD(0) : v(S_t) \leftarrow v(S_t) + \alpha(R_{t+1} + \gamma v(S_{t+1}) - v(S_t))$$



Temporal Difference (1-step to n-step)

On-policy control: SARSA

Estimating action value function Q using TD

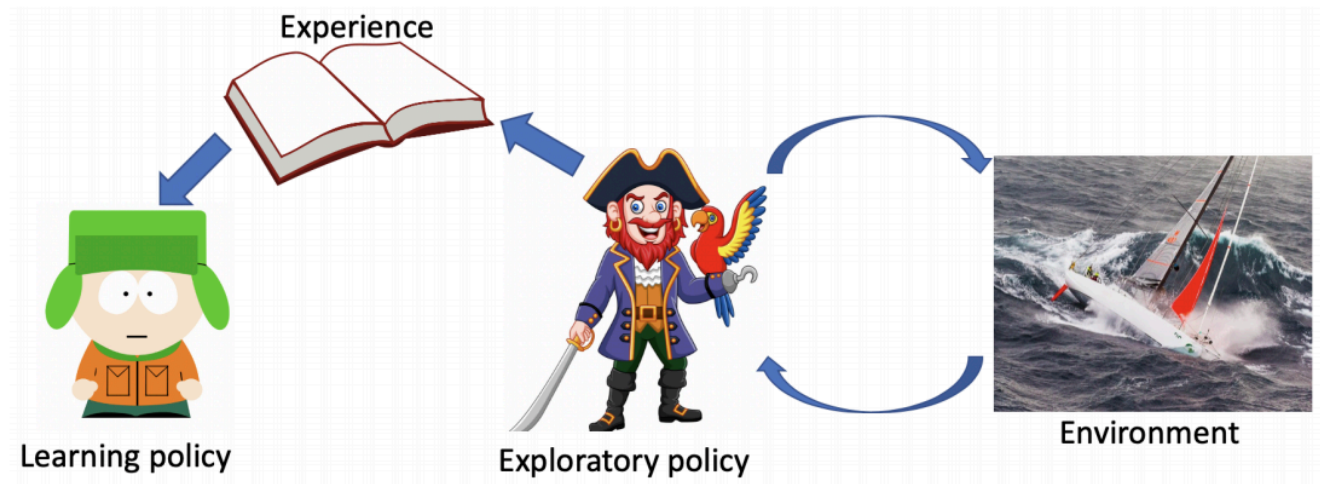
ϵ -greedy policy for one step, then bootstrap the action value function:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right]$$

The update is done after every transition from a nonterminal state S_t

TD target $\delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$

Off-policy learning



- 1 Following behaviour policy $\mu(a|s)$ to collect data

$$S_1, A_1, R_2, \dots, S_T \sim \mu$$

Update π using $S_1, A_1, R_2, \dots, S_T$

- 2 It leads to many benefits:
 - 1 Learn about optimal policy while following exploratory policy
 - 2 Learn from observing humans or other agents
 - 3 Re-use experience generated from old policies $\pi_1, \pi_2, \dots, \pi_{t-1}$

Off-policy control: Q-Learning

- 1 The target learning policy π is **greedy** on $Q(s, a)$

$$\pi(s) = \arg \max_{a'} Q(s, a')$$

- 2 The behavior policy could be **totally random**, but we let it improve, thus the behavior policy μ is **ϵ -greedy** on $Q(s, a)$
- 3 Thus Q-learning target:

$$\begin{aligned} R_{t+1} + \gamma Q(S_{t+1}, A') &= R_{t+1} + \gamma Q(S_{t+1}, \arg \max_{a'} Q(S_{t+1}, a')) \\ &= R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') \end{aligned}$$

- 4 Thus the Q-Learning update becomes

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

Comparison of Sarsa and Q-Learning

① Sarsa: On-Policy TD control

Choose action A_t from S_t using policy derived from Q with ϵ -greedy

Take action A_t , observe R_{t+1} and S_{t+1}

Choose action A_{t+1} from S_{t+1} using policy derived from Q with ϵ -greedy

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

② Q-Learning: Off-Policy TD control

Choose action A_t from S_t using policy derived from Q with ϵ -greedy

Take action A_t , observe R_{t+1} and S_{t+1}

Then 'imagine' A_{t+1} as $\arg \max_a Q(S_{t+1}, a')$ in the update target

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

Outline

- Basics of RL
- MDP and tabular solution methods
- **Value function approximation**
- Policy optimization
- Other topics: model-based RL, imitation learning, RL system design
- Open problems

Scaling Up RL: Function Approximation

- Previous tabular methods only work for RL problems with small number of states (lookup table representation)
- Many real-world problems have a large number of states
- Represent a value function with a parameterized function

$$\hat{v}(s, \mathbf{w}) \approx v_{\pi}(s)$$

$$\hat{q}(s, a, \mathbf{w}) \approx q_{\pi}(s, a)$$

$$\hat{\pi}(a, s, \mathbf{w}) \approx \pi(a|s)$$

Scaling Up RL: Value Function Approximation

- 1 Approximate the action-value function

$$\hat{q}(S, A, \mathbf{w}) \approx q_{\pi}(S, A)$$

- 2 Minimize the MSE (mean-square error) between approximate action-value and true action-value (assume oracle)

$$J(\mathbf{w}) = \mathbb{E}_{\pi}[(q_{\pi}(S, A) - \hat{q}(S, A, \mathbf{w}))^2]$$

- 3 Stochastic gradient descend to find a local minimum

$$\Delta \mathbf{w} = \alpha(q_{\pi}(S, A) - \hat{q}(S, A, \mathbf{w}))\nabla_{\mathbf{w}}\hat{q}(S, A, \mathbf{w})$$

Scaling Up RL: Value Function Approximation

Same to the prediction, there is no oracle for the true value $q_\pi(S, A)$, so we substitute a target

- 1 For MC, the target is the return G_t

$$\Delta \mathbf{w} = \alpha (\mathbf{G}_t - \hat{q}(S_t, A_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(S_t, A_t, \mathbf{w})$$

- 2 For Sarsa, the target is the TD target $R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w})$

$$\Delta \mathbf{w} = \alpha (\mathbf{R}_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(S_t, A_t, \mathbf{w})$$

- 3 For Q-learning, the target is the TD target

$$R_{t+1} + \gamma \max_a \hat{q}(S_{t+1}, a, \mathbf{w})$$

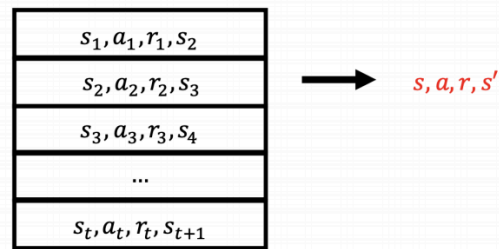
$$\Delta \mathbf{w} = \alpha (\mathbf{R}_{t+1} + \gamma \max_a \hat{q}(S_{t+1}, a, \mathbf{w}) - \hat{q}(S_t, A_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{q}(S_t, A_t, \mathbf{w})$$

Deep Q Learning

- Two of the issues causing problems:
 - Correlations between samples
 - Non-stationary targets
- Deep Q-learning (DQN) addresses both challenges by
 - Experience replay
 - Fixed Q targets

Deep Q Learning: Experience Replay

- 1 To reduce the correlations among samples, store transition (s_t, a_t, r_t, s_{t+1}) in replay memory $\mathcal{D}$



- 2 To perform experience replay, repeat the following
 - 1 sample an experience tuple from the dataset: $(s, a, r, s') \sim \mathcal{D}$
 - 2 compute the target value for the sampled tuple: $r + \gamma \max_{a'} \hat{Q}(s', a', \mathbf{w})$
 - 3 use stochastic gradient descent to update the network weights

$$\Delta \mathbf{w} = \alpha \left(r + \gamma \max_{a'} \hat{Q}(s', a', \mathbf{w}) - Q(s, a, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{Q}(s, a, \mathbf{w})$$

Deep Q Learning: Fixed Targets

- ① To help improve stability, fix the target weights used in the target calculation for multiple updates
- ② Let a different set of parameter $\mathbf{w}^-$ be the set of weights used in the target, and $\mathbf{w}$ be the weights that are being updated
- ③ To perform experience replay with fixed target, repeat the following
 - ① sample an experience tuple from the dataset: $(s, a, r, s') \sim \mathcal{D}$
 - ② compute the target value for the sampled tuple:
 $r + \gamma \max_{a'} \hat{Q}(s', a', \mathbf{w}^-)$
 - ③ use stochastic gradient descent to update the network weights

$$\Delta \mathbf{w} = \alpha \left(r + \gamma \max_{a'} \hat{Q}(s', a', \mathbf{w}^-) - Q(s, a, \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{Q}(s, a, \mathbf{w})$$



Q target



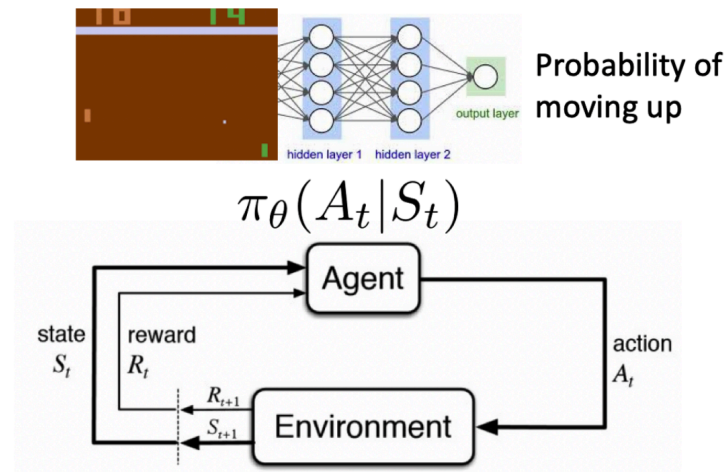
Q estimation

Outline

- Basics of RL
- MDP and tabular solution methods
- Value function approximation
- **Policy optimization**
- Model-based RL, imitation learning, RL system
- Open problems

Policy Optimization

- Value-based RL: learn value function first, then make decision based on action value function
- Policy-based RL: optimize the policy



Policy Optimization

① Value-based RL

- ① to learn value function
- ② implicit policy based on the value function

② Policy-based RL

- ① no value function
- ② to learn policy directly

③ Actor-critic

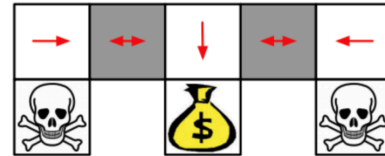
- ① to learn both policy and value function

- ① In value-based RL deterministic policy is generated directly from the value function using greedy $a_t = \arg \max_a Q(a, s_t)$
- ② In policy optimization, we have stochastic policy output from $\pi_\theta(a|s)$ where θ is the policy parameter to optimize

Policy Optimization: Stochastic Policy



- ① Two-player game
- ② What is the best policy?
 - ① A deterministic policy is easily beaten
 - ② Uniform random policy is the optimal (Nash equilibrium)



- ① Policy-based RL can learn the optimal stochastic policy
- ② An optimal stochastic policy will randomly move E or W in two grey states

$$\pi_{\theta}(\text{wall to N and W, move E}) = 0.5$$

$$\pi_{\theta}(\text{wall to N and W, move W}) = 0.5$$

Policy Optimization: Objective

$$\theta^* = \arg \max_{\theta} J(\theta) \text{ where } J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [r(\tau)]$$

If $J(\theta)$ is differentiable, we can use gradient-based methods

- Gradient ascend
- Conjugate gradient
- quasi-newton

Some derivative-free black-box optimization methods:

- Cross-entropy method (CEM)
- Hill climbing
- Evolution algorithm

Policy Optimization: Objective

- Policy optimization objective:

$$\theta^* = \arg \max_{\theta} J(\theta) = \arg \max_{\theta} \sum_{\tau} P(\tau; \theta) R(\tau)$$

- Policy gradient:

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \nabla_{\theta} \sum_{\tau} P(\tau; \theta) R(\tau) \\ &= \sum_{\tau} \nabla_{\theta} P(\tau; \theta) R(\tau) \\ &= \sum_{\tau} \frac{P(\tau; \theta)}{P(\tau; \theta)} \nabla_{\theta} P(\tau; \theta) R(\tau) \\ &= \sum_{\tau} P(\tau; \theta) R(\tau) \frac{\nabla_{\theta} P(\tau; \theta)}{P(\tau; \theta)} \\ &= \sum_{\tau} P(\tau; \theta) R(\tau) \nabla_{\theta} \log P(\tau; \theta) \end{aligned}$$

log derivative trick

Policy Optimization: Decomposition

- 1 Approximate with empirical estimate for m sample paths under policy π_θ :

$$\nabla_\theta J(\theta) \approx \frac{1}{m} \sum_{i=1}^m R(\tau_i) \nabla_\theta \log P(\tau_i; \theta)$$

- 2 Decompose $\nabla_\theta \log P(\tau; \theta)$

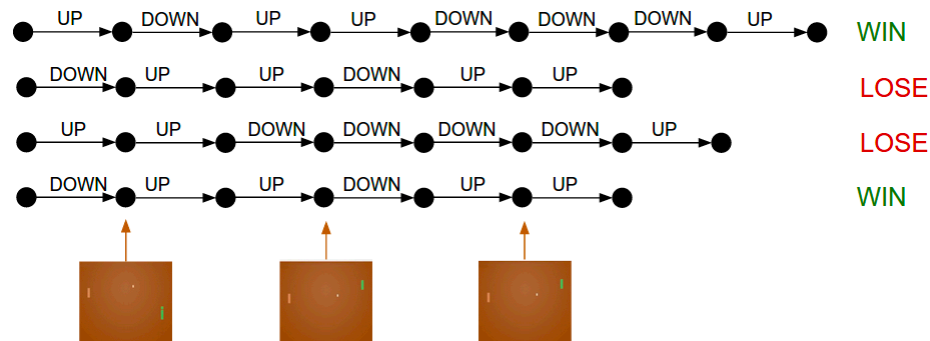
$$\begin{aligned} \nabla_\theta \log P(\tau; \theta) &= \nabla_\theta \log \left[\mu(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t | s_t) p(s_{t+1} | s_t, a_t) \right] \\ &= \nabla_\theta \left[\log \mu(s_0) + \sum_{t=0}^{T-1} \log \pi_\theta(a_t | s_t) + \log p(s_{t+1} | s_t, a_t) \right] \\ &= \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t | s_t) \end{aligned}$$

Policy Optimization

- So we have the following approximate gradient from several simulations

$$\nabla_{\theta} J(\theta) \approx \frac{1}{m} \sum_{i=1}^m R(\tau_i) \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i)$$

Unbiased but very noisy



Reduce Variance of Policy Gradient

- Causality: policy at time t' cannot affect the reward at time $t < t'$

$$\nabla_{\theta} \mathbb{E}[R] \approx \frac{1}{m} \sum_{i=1}^m \sum_{t=0}^{T-1} G_t^{(i)} \cdot \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i)$$

- Baseline to reduce the variance

$$\nabla_{\theta} \mathbb{E}_{\tau \sim \pi_{\theta}}[R] = \mathbb{E}_{\tau} \left[\sum_{t=0}^{T-1} (G_t - b(s_t)) \cdot \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

$$b(s_t) = \mathbb{E}[r_t + r_{t+1} + \dots + r_{T-1}]$$

REINFORCE: A Monte-Carlo policy gradient

- Basic policy optimization algorithm:

REINFORCE, A Monte-Carlo Policy-Gradient Method (episodic)

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$

Repeat forever:

 Generate an episode $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \theta)$

 For each step of the episode $t = 0, \dots, T - 1$:

$G \leftarrow$ return from step t

$\theta \leftarrow \theta + \alpha \gamma^t G \nabla_{\theta} \ln \pi(A_t|S_t, \theta)$

Reducing Variance using a Critic

- Previously G is a sample from Monte Carlo simulation, which is unbiased but noisy estimate of $Q(s, a)$

$$\nabla_{\theta} \mathbb{E}_{\pi_{\theta}}[R] = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^{T-1} \mathbf{G}_t \cdot \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

- Instead we can use a critic to estimate the action-value function

$$\nabla_{\theta} \mathbb{E}_{\pi_{\theta}}[R] = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^{T-1} \mathbf{Q}_w(s_t, a_t) \cdot \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

Actor-Critic Policy Gradient

- Actor-critic policy gradient
 - Actor: the policy function used to generate the action
 - Critic: the value function used to evaluate the reward of the actions
- Actor-critic algorithms maintain two sets of parameters
 - Actor: Updates policy parameters θ , in direction suggested by critic
 - Critic: Updates action-value function parameters w

$$\nabla_{\theta} \mathbb{E}_{\pi_{\theta}}[R] = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^{T-1} Q_{\mathbf{w}}(s_t, a_t) \cdot \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

Actor-Critic Policy Gradient

- ① Using a linear value function approximation: $Q_{\mathbf{w}}(s, a) = \psi(s, a)^T \mathbf{w}$
 - ① **Critic**: update $\mathbf{w}$ by a linear TD(0)
 - ② **Actor**: update θ by policy gradient

Algorithm 1 Simple QAC

```
1: for each step do
2:   generate sample  $s, a, r, s', a'$  following  $\pi_\theta$ 
3:    $\delta = r + \gamma Q_{\mathbf{w}}(s', a') - Q_{\mathbf{w}}(s, a)$     #TD error
4:    $\mathbf{w} \leftarrow \mathbf{w} + \beta \delta \psi(s, a)$ 
5:    $\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(s, a) Q_{\mathbf{w}}(s, a)$ 
6: end for
```

Advantage Actor-Critic Policy Gradient

- Advantage function:

$$A^{\pi,\gamma}(s, a) = Q^{\pi,\gamma}(s, a) - V^{\pi,\gamma}(s)$$

- Value function serves as a baseline for Q

$$\begin{aligned} V^{\pi,\gamma}(s) &= \mathbb{E}_{\pi}[r_1 + \gamma r_2 + \dots | s_1 = s] \\ &= \mathbb{E}_{a \sim \pi}[Q^{\pi,\gamma}(s, a)] \end{aligned}$$

- Then the policy gradient becomes:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}}[\nabla_{\theta} \log \pi_{\theta}(s, a) A^{\pi,\gamma}(s, a)]$$

Example code:

<https://github.com/cuhkrlcourse/RExample/blob/master/policygradient/ac-pong-pytorch.py>

Different variants of policy gradient

- 1 The policy gradient has many forms

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) \mathbf{G}_t] \text{ — REINFORCE} \\ &= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) \mathbf{Q}^{\mathbf{w}}(s, a)] \text{ — Q Actor-Critic} \\ &= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) \mathbf{A}^{\mathbf{w}}(s, a)] \text{ — Advantage Actor-Critic} \\ &= \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(s, a) \delta] \text{ — TD Actor-Critic}\end{aligned}$$

- 2 Critic uses policy evaluation (e.g. MC or TD learning) to estimate $Q^{\pi}(s, a)$, $A^{\pi}(s, a)$, or $V^{\pi}(s, a)$

State of the art Policy Optimization

① Policy Gradient→Natural PG/TRPO→ACKTR→PPO

- ① **TRPO**: Trust region policy optimization. Schulman, L., Moritz, Jordan, Abbeel. 2015
- ② **ACKTR**: Scalable trust-region method for deep reinforcement learning using Kronecker-factored approximation. Y. Wu, E. Mansimov, S. Liao, R. Grosse, and J. Ba. 2017
- ③ **PPO**: Proximal policy optimization algorithms. Schulman, Wolski, Dhariwal, Radford, Klimov. 2017

② Q-learning→DDPG→TD3→SAC

- ① **DDPG**: Deterministic Policy Gradient Algorithms, Silver et al. 2014
- ② **TD3**: Addressing Function Approximation Error in Actor-Critic Methods, Fujimoto et al. 2018
- ③ **SAC**: Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor, Haarnoja et al. 2018

Log Derivative Trick vs. Reparameterization Trick

Problem: $\nabla_{\theta} \mathbb{E}_{x \sim p_{\theta}(x)} [f(x)]$

Log Derivative Trick:

$$\begin{aligned}\nabla_{\theta} \mathbb{E}_{x \sim p_{\theta}(x)} [f(x)] &= \nabla_{\theta} \int f(x) p_{\theta}(x) dx \\ &= \int f(x) \nabla_{\theta} p_{\theta}(x) dx \\ &= \int f(x) p_{\theta}(x) \nabla_{\theta} \log p_{\theta}(x) dx \\ &= \mathbb{E}_{x \sim p_{\theta}(x)} [f(x) \nabla_{\theta} \log p_{\theta}(x)] \\ &\approx \frac{1}{N} \sum_{i=1}^N f(x_i) \nabla_{\theta} \log p_{\theta}(x_i)\end{aligned}$$

Reparameterization Trick:

$$\begin{aligned}\varepsilon &\sim q(\varepsilon) \\ x &= g_{\theta}(\varepsilon) \\ \nabla_{\theta} \mathbb{E}_{x \sim p_{\theta}(x)} [f(x)] &= \nabla_{\theta} \mathbb{E}_{\varepsilon \sim q(\varepsilon)} [f(g_{\theta}(\varepsilon))] \\ &= \mathbb{E}_{\varepsilon \sim q(\varepsilon)} [\nabla_{\theta} f(g_{\theta}(\varepsilon))] \\ &\approx \frac{1}{N} \sum_{i=1}^N (\nabla_{\theta} f(g_{\theta}(\varepsilon_i)))\end{aligned}$$

Properties	REINFORCE	Reparameterization
Differentiability requirements	Can work with a non-differentiable model	Needs a differentiable model
Gradient variance	High variance; needs variance reduction techniques	Low variance due to implicit modeling of dependencies
Type of distribution	Works for both discrete and continuous distributions	In the current form, only valid for continuous distributions
Family of distribution	Works for a large class of distributions of x	It should be possible to reparameterize x as done above

Log Derivative Trick:

$$\begin{aligned}
 \nabla_{\theta} \mathbb{E}_{x \sim p_{\theta}(x)} [f(x)] &= \nabla_{\theta} \int f(x) p_{\theta}(x) dx \\
 &= \int f(x) \nabla_{\theta} p_{\theta}(x) dx \\
 &= \int f(x) p_{\theta}(x) \nabla_{\theta} \log p_{\theta}(x) dx \\
 &= \mathbb{E}_{x \sim p_{\theta}(x)} [f(x) \nabla_{\theta} \log p_{\theta}(x)] \\
 &\approx \frac{1}{N} \sum_{i=1}^N f(x_i) \nabla_{\theta} \log p_{\theta}(x_i)
 \end{aligned}$$

Reparameterization Trick:

$$\begin{aligned}
 \varepsilon &\sim q(\varepsilon) \\
 x &= g_{\theta}(\varepsilon) \\
 \nabla_{\theta} \mathbb{E}_{x \sim p_{\theta}(x)} [f(x)] &= \nabla_{\theta} \mathbb{E}_{\varepsilon \sim q(\varepsilon)} [f(g_{\theta}(\varepsilon))] \\
 &= \mathbb{E}_{\varepsilon \sim q(\varepsilon)} [\nabla_{\theta} f(g_{\theta}(\varepsilon))] \\
 &\approx \frac{1}{N} \sum_{i=1}^N (\nabla_{\theta} f(g_{\theta}(\varepsilon_i)))
 \end{aligned}$$

Great resource on deep RL

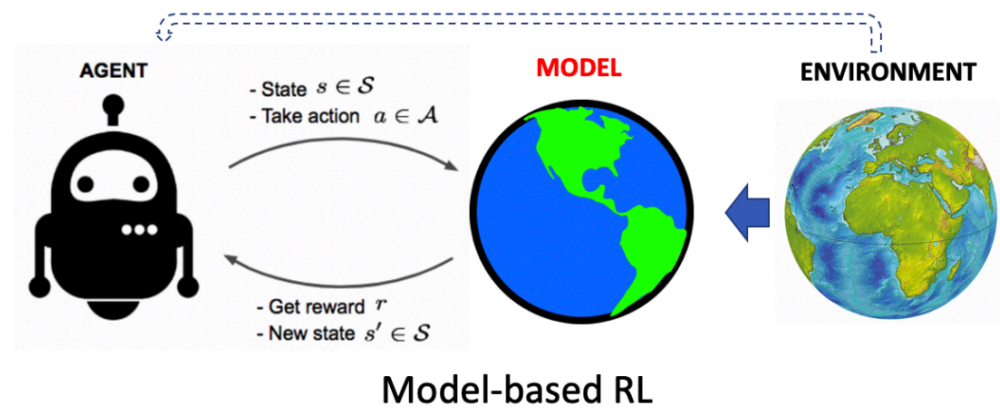
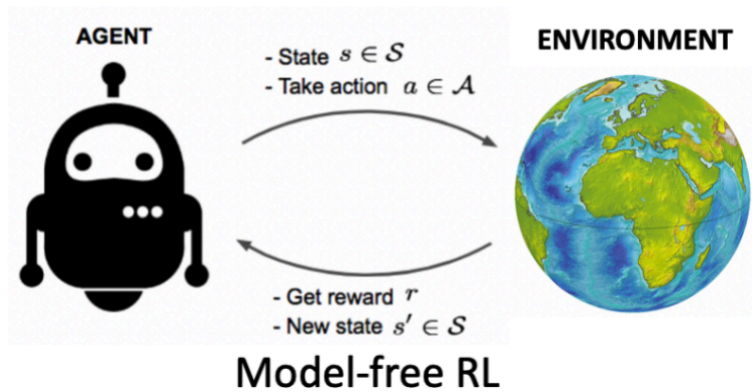
- Spinning-Up: Nice implementations and summary of the algorithms from OpenAI:
 - <https://spinningup.openai.com>
- Nuts and Bolts of deep RL experimentation by John Schulman
 - <https://www.youtube.com/watch?v=8EcdaCk9KaQ>
 - <http://joschu.net/docs/nuts-and-bolts.pdf>



Outline

- Basics of RL
- MDP and tabular solution methods
- Value function approximation
- Policy optimization
- Other topics: model-based RL, imitation learning, RL system
- Open problems

Model-based RL



Model-based RL

- Advantage: better sample efficiency



- Disadvantage: two sources of approximation error (model estimation and function approximation)

Model-based RL

- Model-based value-based RL

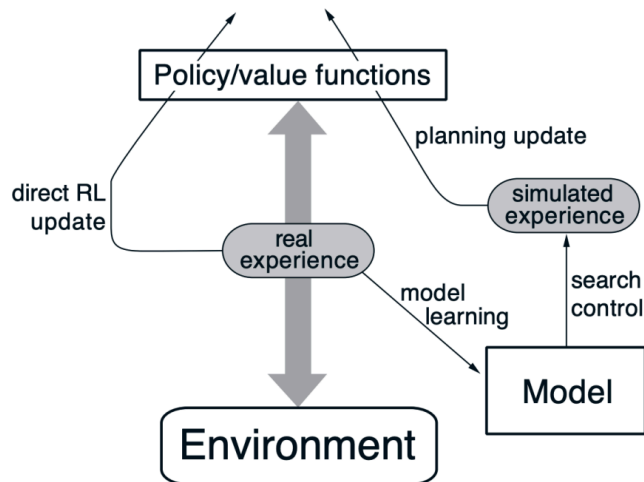
model $\rightarrow$ simulated environment $\xrightarrow{\text{backups}}$ values $\rightarrow$ policy

- Model-based policy-based RL

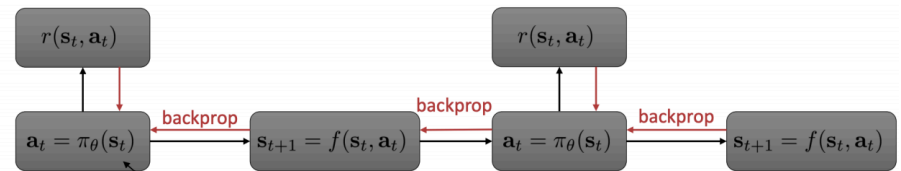
model $\xrightarrow{\text{improves}}$ policy

Model-based RL

Value-based RL: Dyna



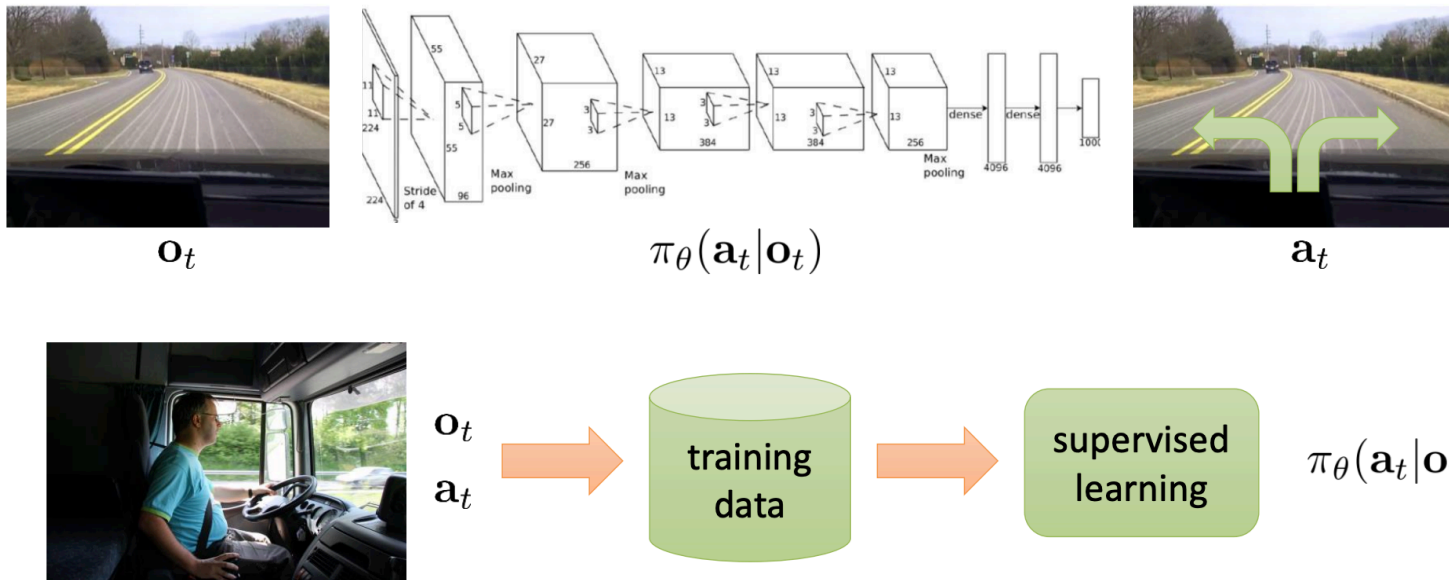
Policy-based RL: model learning from trajectory optimization



Many robotic applications

Imitation Learning

Supervised learning of the policy network



Unifying IL and RL

Imitation Learning

- Requires demonstrations
- Issue of distributional shift
- Simple stable supervised learning
- Only as good as the demo

Reinforcement Learning

- Requires reward function
- Must address exploration
- Potentially non-convergent
- Can become superhuman good

Can we get the best of both worlds?

What if we can have both demonstrations and rewards

Unifying IL and RL

- Pretrain with IL then finetune with RL

- IL as auxiliary loss function

1. collected demonstration data $(\mathbf{s}_i, \mathbf{a}_i)$
2. initialize π_θ as $\max_\theta \sum_i \log \pi_\theta(\mathbf{a}_i | \mathbf{s}_i)$
3. run π_θ to collect experience
4. improve π_θ with any RL algorithm



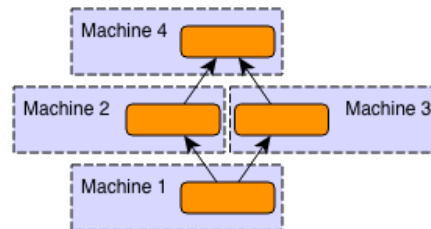
$$\underbrace{E_{\pi_\theta}[r(\mathbf{s}, \mathbf{a})]}_{\text{RL}} + \lambda \underbrace{\sum_{(\mathbf{s}, \mathbf{a}) \in \mathcal{D}_{\text{demo}}} \log \pi_\theta(\mathbf{a} | \mathbf{s})}_{\text{Imitation learning}}$$

- Off-policy reinforcement learning
 - Policy gradient with important sampling
 - Q-learning (put demonstration in replay buffer)

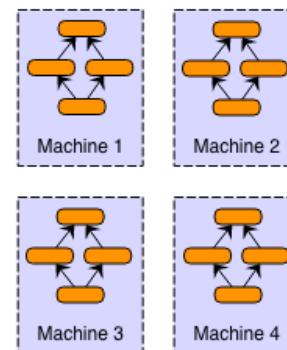
Distributed ML Systems and RL Systems

- **Model parallelism:** different machines are responsible for computation in different parts of a single network
- **Data parallelism:** different machine has a complete copy of the model, each machine gets a different portion of the data

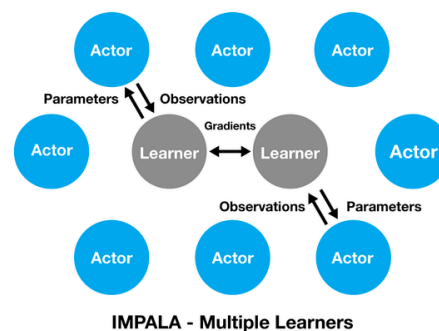
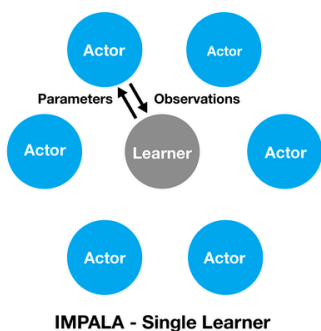
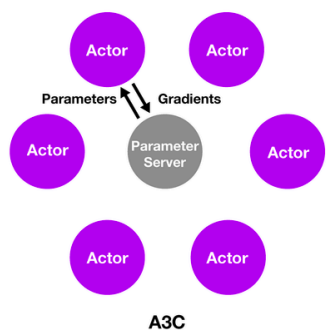
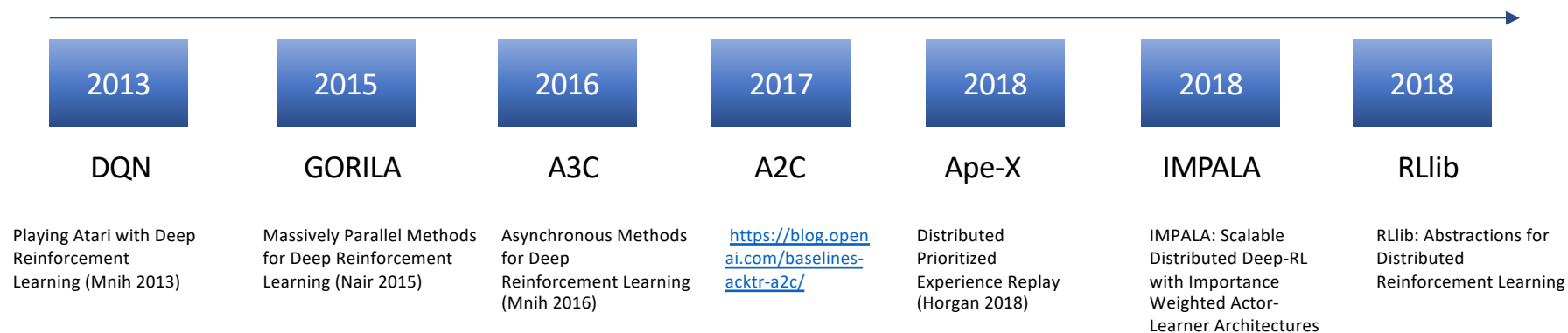
Model Parallelism



Data Parallelism



Distributed ML Systems and RL Systems



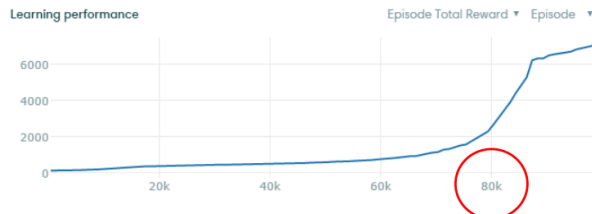
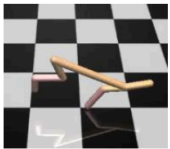
Outline

- Basics of RL
- MDP and tabular solution methods
- Value function approximation
- Policy optimization
- Other topics: model-based RL, imitation learning, RL system design
- **Open problems**

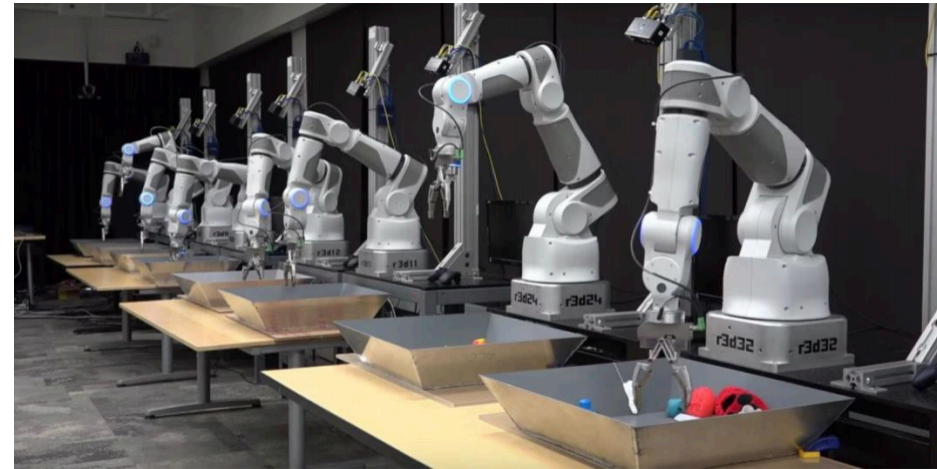
Open Problems in RL

Sample efficiency

Simulation: relatively easy to collect sample



Real-world problems: high cost to collect samples



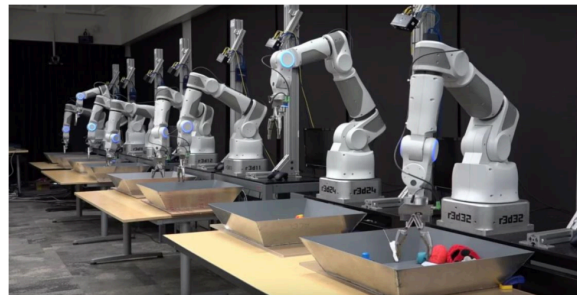
Open Problems in RL

Generalist RL rather than specialist RL

- Prior knowledge?
- Multi-task learning?



Pinto & Gupta, 2015



Levine et al. 2016



Open Problems in RL

New environments and agent designs

OpenAI Gym

MuJoCo

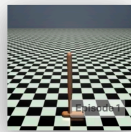
Continuous control tasks, running in a fast physics simulator.



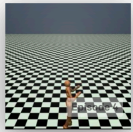
Ant-v2
Make a 3D four-legged robot walk.



HalfCheetah-v2
Make a 2D cheetah robot run.



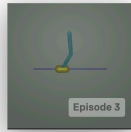
Hopper-v2
Make a 2D robot hop.



Humanoid-v2
Make a 3D two-legged robot walk.



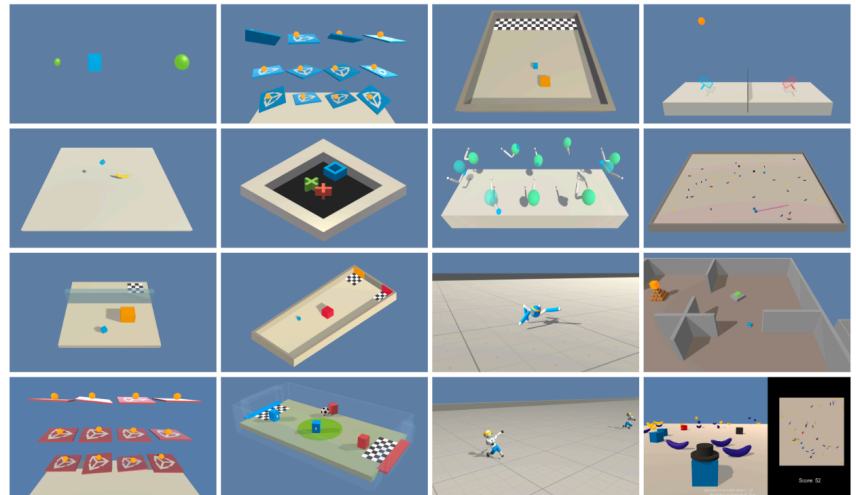
HumanoidStandup-v2
Make a 3D two-legged robot stand up.



InvertedDoublePendulum-v2
Balance a pole on a pole on a cart.

https://gym.openai.com/envs/#classic_control

Define your problem, and develop your own environment!



<https://github.com/Unity-Technologies/ml-agents>

Open Problems in RL

Bridge RL with other ML topics (predictive learning, generative models, etc)

Yann LeCun's cake

How Much Information Does the Machine Need to Predict? Y LeCun

- **"Pure" Reinforcement Learning (cherry)**
 - ▶ The machine predicts a scalar reward given once in a while.
 - ▶ **A few bits for some samples**
- **Supervised Learning (icing)**
 - ▶ The machine predicts a category or a few numbers for each input
 - ▶ Predicting human-supplied data
 - ▶ **10→10,000 bits per sample**
- **Unsupervised/Predictive Learning (cake)**
 - ▶ The machine predicts any part of its input for any observed part.
 - ▶ Predicts future frames in videos
 - ▶ **Millions of bits per sample**



RL Applications

pricing, trading
portfolio opt.
risk mgmt

finance

recommendation
e-commerce, OR
customer mgmt

business
management

adaptive
decision
control

energy

adaptive
traffic signal
control

transportation

proficiency est.
recommendation
education games

education

DTRs
diagnosis
EHR/EMR

healthcare

games

Go, poker
Dota, bridge
Starcraft

robotics

sim-to-real
co-robot
control

NLP

seq. gen.
translation
dialog, QA, IE, IR

computer
vision

recognition
detection
perception

computer
systems

topics in
computer
science

science
engineering
art

maths, physics,
chemistry, music
drawing, animation

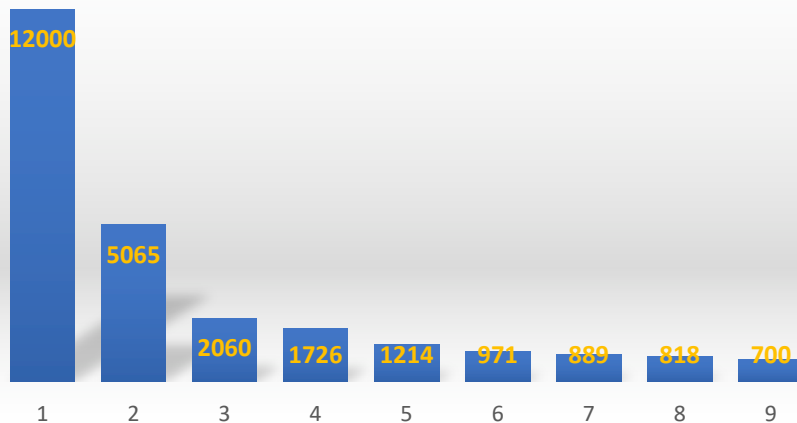
<https://arxiv.org/pdf/1908.06973.pdf>

May ReinForce Be With You!

<https://github.com/zhoubolei/introRL>



每课浏览量 - 截止2020年5月24日



Lecture 1	Overview
Lecture 2	Markov Decision Process
Lecture 3	Model-free Prediction and Control
Lecture 4	Value Function Approximation
Lecture 5	Policy Optimization: Foundation
Lecture 6	Policy Optimization: State of the art
Lecture 7	Model-based RL
Lecture 8	Imitation Learning
Lecture 9	Distributed systems for RL
Lecture 10	RL in a nutshell

What's Next? 番外篇 ?

- Generative modeling
- Unsupervised representation learning
- Advanced topics of RL:
 - Go game bot: AlphaGo Series
 - Starcraft2 game bot: AlphaStar
 - Dota2 game bot: OpenAI Five
 - MetaRL and multitasking
 - 欢迎留言建议
- 强化学习教材课本?